🌐 Network Layer: Services and Protocols

What the Network Layer Does

- Its job: **Deliver data from one host to another** (host-to-host communication).
- It sits between **Transport Layer (segments)** and **Link Layer (frames)**.

At the sender side:

- Takes a **transport segment** (from TCP/UDP).
- Encapsulates it into a **datagram (IP packet)**.
- Hands it to the link layer for transmission.

At the receiver side:

- Gets the datagram from the link layer.
- Extracts the **segment** and passes it up to the transport layer.

Where the Network Layer Lives

- Runs on **every Internet device**:
  - **End hosts** (sender and receiver).
  - **Routers** (in between).

Routers' role:

- Don't care about app or transport data.
- They only **look at IP headers** and decide where to forward the packet next.

🚦 Two Main Functions

1. **Forwarding** (local action)
   - Move a packet from a router's **input port → correct output port**.
   - Analogy: *Going through a single traffic intersection.*
2. **Routing** (global action)
   - Decide the **end-to-end path** a packet should take across the network.
   - Done using **routing algorithms** (like shortest path, OSPF, BGP).
   - Analogy: *Planning your trip from New York to LA before starting.*

🗺️ Analogy: Taking a Road Trip

- **Routing** = choosing the entire route on Google Maps (planning).
- **Forwarding** = at each intersection, turning left/right as per the map (local move).

☑ **Quick Recap (Exam Style):**

- **Encapsulation**: segment → datagram.
- **Forwarding** = per-router decision (input → output).
- **Routing** = overall path selection.
- Network layer is implemented **in every host + every router**.
- Internet's network layer protocol = **IP (IPv4/IPv6)**.

🌐 Network Layer: Data Plane vs Control Plane

Think of a **router** as having two "brains":

🚦 Data Plane (local, per router)

- **Scope**: Works **inside a single router**.
- **Job**: For each arriving packet → **decide output port and forward it**.
- **Based on**: Packet's header values (like destination IP).
- **Analogy**: A traffic cop at an intersection, directing one car at a time.

🧭 Control Plane (network-wide)

- **Scope**: Works **across the entire network**.
- **Job**: Decide the **overall path** packets should follow (routing).
- **Analogy**: Google Maps planning your whole trip before you drive.

🔑 Two Approaches to the Control Plane

1. **Traditional routing (per-router)**
    - Each router runs its own **routing algorithm** (like OSPF, RIP, BGP).
    - Routers talk to each other, share info, and compute routes.
    - Routing logic is *distributed*.
2. **Software-Defined Networking (SDN)**
    - Centralized control.
    - Routing logic is in a **remote controller (server)**.
    - Routers just forward packets based on rules given by the controller.

🗂️ Per-Router Control Plane

- Each router has:
    - **Data plane**: forwards packets.
    - **Control plane module**: runs routing algorithm, updates forwarding table.
- The control plane in *all routers together* ensures consistent paths.

☑ **Quick Recap (Exam Style):**

- **Data plane** = local forwarding (per packet, per router).
- **Control plane** = global routing logic (whole path planning).
- **Traditional** = routing done in each router (distributed).
- **SDN** = routing done by a central controller (remote).

🌐 1. Connection-less Service (Datagram Network)

- **Example**: The **Internet (IP)**
- **How it works**:
    - Each packet is independent (like sending postcards).
    - Packet has the full **destination address**.
    - Routers forward packets individually — they don't "remember" past packets.
- **Properties**:
    - **No call setup** required.
    - Routers keep **no connection state**.
    - May arrive **out of order** or even get lost.
- **Analogy**: Dropping letters into a postal mailbox — each one is delivered on its own.

🌐 2. Connection Service (Virtual Circuit Network)

- **Example**: Old systems like **ATM, Frame Relay, X.25**
- **How it works**:
  - A **call setup** happens before data flows (like dialing a phone call).
  - Every packet carries a **VC identifier** (not full destination address).
  - Routers along the path keep **state information** (forwarding entries for that VC).
  - Resources (bandwidth, buffers) can be reserved → more predictable service.
- **Properties**:
  - More reliable, controlled.
  - Extra overhead: call setup + teardown.
- **Analogy**: Making a telephone call — first set up, then talk, then hang up.

🔑 Implementation Differences

- **Datagram (IP)**: forwarding tables map **destination addresses → output links**.
- **VC networks**: forwarding tables map **VC numbers → output VC numbers + links**.

✨ Easy Exam Recap

- **Datagram (connectionless)** = Internet today (IP).
- **Virtual Circuits (connection-oriented)** = ATM, X.25, telephone-like.
- **Main difference**:
  - Datagram = no setup, no state, each packet independent.
  - VC = setup/teardown, routers keep state, predictable service.

🌐 Virtual Circuits (VC) — Explained Simply

🔹 What is it?

- A **connection-oriented service at the network layer**.

- Works like a **telephone call**:
  - **Setup first**, then data flows, then **teardown**.
- **Not used in today's Internet (IP is connectionless)**, but important for understanding ATM, Frame Relay, X.25.

🔹 How It Works

1. **Call setup** (path chosen, resources may be reserved).
2. **Each packet** carries a **VC identifier** (short number, not full destination address).
3. **Routers** along the path:
   - Maintain **state information** about each VC.
   - Use a **forwarding table**: maps *incoming VC# + interface* → *outgoing VC# + interface*.
4. **Call teardown** when the session ends.

🔹 Key Features

- **VC Identifier (VCI)**: replaces full destination address in packet.
- **Resources (bandwidth, buffers)** may be reserved → predictable performance.
- **Routers maintain state** → need memory, can scale less easily than IP.
- VC numbers can be **renumbered at each hop** (like "flight numbers" changing between airports).

🔹 VC Forwarding Table (Example)

Incoming Interface	Incoming VC#	Outgoing Interface	Outgoing VC#
1	12	3	22
2	63	1	18
3	7	2	17

👉 This tells the router: "If a packet arrives on interface 1 with VC# 12, forward it out interface 3 with VC# 22."

🔹 VC Signaling Protocol (setup steps)

1. Initiate call

2. Incoming call
3. Accept call
4. Call connected
5. Data flow begins
6. Call teardown

🔑 Quick Exam Recap

- **Datagram network (IP)** → no setup, no state, each packet has full address.
- **VC network** → setup/teardown, routers keep state, packets carry short VC number.
- **Analogy**:
  - Datagram = postal service (every letter has full address).
  - VC = telephone call (set up once, then just talk).

📊 Datagram Networks vs Virtual Circuits (VCs)

Feature	Datagram Network (e.g., Internet / IP)	Virtual Circuit (e.g., ATM, X.25)
Setup	❌ No call setup — packets can start anytime	☑ Requires call setup before sending data
Router State	❌ Routers keep **no per-flow state** (stateless)	☑ Routers keep **per-connection state** (VC tables)
Packet Header	Contains **full destination address**	Contains **short VC identifier (VCI)**
Forwarding	Router looks up destination address in **forwarding table** (using longest prefix match)	Router looks up **VC# in VC forwarding table** to find outgoing link/VC#
Resource Allocation	❌ Best-effort, no guarantees	☑ Can reserve bandwidth/buffers for predictable service
Failure Handling	Each packet routed independently → rerouting possible	VC broken if path fails (connection must be re-established)
Analogy	📬 Postal service → every letter has full address	📞 Telephone call → set up once, then just talk

---

🔹 Datagram Forwarding (example)

- Each router uses the **IP destination address**.

- Forwarding table stores **address ranges** (not all 4 billion addresses individually). Example:
- `Address range 1 → output link 3`
- `Address range 2 → output link 2`
- `Address range 3 → output link 2`
- `Address range 4 → output link 1`

So when a datagram arrives, the router checks:
"Which range does this destination IP fall into?" → forwards accordingly.

☑ Exam Trick to Remember

- **Datagram/IP** = *Stateless, flexible, scalable* (good for the Internet).
- **VC** = *Stateful, predictable, telephone-like* (good for guaranteed service, but less scalable).

Datagram Networks (like the Internet)

1. **No call setup**
   - Unlike a phone call (VC), datagram networks don't need setup.
   - You just send a packet, and it goes on its way.
2. **Routers are stateless**
   - Routers don't remember "connections."
   - They only care about the **current packet** and its **destination address**.
3. **Packets independent**
   - Each packet carries the full **IP destination address**.
   - Packets can even take different paths to the destination.

How Forwarding Works

Imagine a router receiving a packet:

- It looks at the **destination IP address** in the header.
- It consults its **forwarding table** (built using routing algorithms).
- The table doesn't list every single IP (too many!). Instead, it lists **address ranges** → output links.

Example forwarding table:

Address Range	Output Link
Range 1	3
Range 2	2
Range 3	2
Range 4	1

So if a packet's IP falls into Range 2 → forward to **output link 2**.

✨ Key Idea

- **Scalability**: Instead of storing billions of individual addresses, routers **aggregate addresses into ranges** (like ZIP codes in mail).
- **Analogy**: This is like the postal service — each letter has the full destination address written on it, and each post office forwards it based on **address ranges** (ZIP code prefixes).

👉 Quick check for you:
If every datagram has to carry the **full destination IP address**, what do you think the trade-off is compared to **virtual circuits** that only carry a short VC identifier?

1. **Datagram forwarding with address ranges**
   - Instead of keeping an entry for every one of the ~4 billion IPs, routers store *aggregated ranges* (like you showed).
   - Example:
     - One range maps to interface 0,
     - another range to interface 1, etc.
2. **But… what if ranges overlap or don't divide neatly?**
   - That's the problem: a destination IP might match *more than one range*.

👉 The solution is **Destination-based forwarding with Longest Prefix Matching**:

- Each range can be written as a **prefix** (like `/20`, `/24`, etc.).
- When a router checks a packet's destination address:
  0. It finds all ranges (prefixes) that match.
  1. It chooses the one with the **longest prefix** (most bits fixed).
  2. That's the most specific and accurate rule.

💡 Quick analogy:
Think of mailing addresses.

- Rule 1: "All letters for *New York State* go to box A."
- Rule 2: "All letters for *New York City* go to box B."
- A letter addressed to "New York City" matches *both rules* — but the city rule is more specific, so it should go to box B.

❓ The problem

Routers use **address ranges** (prefixes) in their forwarding tables, but real IP addresses don't always divide neatly into non-overlapping blocks.
So… what happens when an IP address matches **multiple prefixes**?

☑ The solution: Longest Prefix Matching

- A router checks **all prefixes** in its forwarding table that match the packet's destination address.
- If multiple entries match → it chooses the one with the **longest prefix** (the most specific match).
- Think: *the more digits of the ZIP code match, the more accurate the delivery.*

🔎 Example

Forwarding table (prefix → interface):

Destination Prefix	Interface
`11001000 00010111 00010*`	0
`11001000 00010111 00011000*`	1
`11001000 00010111 00011*`	2
otherwise	3

Now test two packets:

1. **`11001000 00010111 00011000 10101010`**
   - Matches prefix `11001000 00010111 00011000*` (length 20).
   - Also matches shorter prefixes, but router picks the **longest**.
   - → Forward to **Interface 1** ☑

2. `11001000 00010111 00010110 10100001`
   - Matches `11001000 00010111 00010*` (length 17).
   - No longer match available.
   - → Forward to **Interface 0** ☑

🧠 Analogy

It's like dialing a phone number:

- If the system finds an area code match (`110`), but also finds a **longer match** (`110-010`), it routes based on the longer one.
- Longer prefix → more precise routing.

👉 Quick check for you:
If a router only supported **shortest prefix match** instead of longest, what problem would that cause in packet delivery?

🌐 Longest Prefix Matching (LPM)

Routers use **prefixes** instead of exact addresses because there are too many IPs to store individually.

- **Forwarding Table (simplified):**

Destination Prefix	Interface
`11001000 00010111 00010***`	0
`11001000 00010111 00011000`	1
`11001000 00010111 00011***`	2
otherwise	3

👉 Rule:
When a packet comes in, match its destination address against these prefixes.
If multiple prefixes match, choose the **longest one** (most bits in common).

- **Example 1:**
  Packet: `11001000 00010111 00011000 10101010`
  Matches prefix `11001000 00010111 00011000` exactly.
  ➡️ Goes to **interface 1**.
- **Example 2:**
  Packet: `11001000 00010111 00010110 10100001`

Matches `11001000 00010111 00010***` and also `11001000 00010111 00011***`.
But `00010***` is the **longer match** here.
➡️ Goes to **interface 0**.

💡 **Key idea:** More bits matched = more specific = better match.

📞 Datagram vs. Virtual Circuit Networks

Datagram (Internet)	Virtual Circuit (ATM, Frame Relay)
No setup needed	Requires setup (like a phone call)
Routers keep **no state**	Routers keep **connection state**
Each packet has full **destination IP**	Packets carry a small **VC ID**
Flexible, "elastic" service (good for computers)	Predictable service (good for voice/video)
Smarter **end systems** (computers handle errors, retransmits)	Simpler **end systems** (phones rely on network)
Complexity at the edge	Complexity in the core

💡 Think of it like this:

- **Datagram network (Internet)** → like sending letters: every envelope has a full address.
- **VC network (ATM/telephone)** → like a phone call: you set up the line once, then just talk without repeating the address.

☑ Quick check for you:
If you had to send **real-time video** (e.g., a Zoom call), which network model — datagram or VC — would you *prefer*? Why?

🌐 Network Layer in the Internet

Key Functions of the Network Layer

1. **Host & Router Roles**
   - **Hosts:** send and receive packets (datagrams) using IP.
   - **Routers:** forward datagrams between networks using **forwarding tables** and **path-selection algorithms**.
2. **Protocols**
   - **IP (Internet Protocol):** main protocol, responsible for addressing and forwarding datagrams.

   - **ICMP (Internet Control Message Protocol):** used for error reporting and network signaling (e.g., "destination unreachable").
3. **Routing & Path Selection**
   - **Routing protocols:** OSPF, BGP → determine end-to-end path across the Internet.
   - **SDN controllers:** can centrally control routing decisions in modern networks.
4. **Interaction with Other Layers**
   - **Transport layer:** TCP, UDP → encapsulated inside IP datagrams.
   - **Link layer & Physical layer:** actual transmission of datagrams over the network.

IP Datagram Format (Simplified)

An **IP datagram** carries data (like TCP/UDP segments) across networks.

Field	Description
Version (ver)	IP version (IPv4 = 4, IPv6 = 6)
Header Length (hlen)	Size of the IP header in bytes
Type of Service (TOS)	Prioritization bits, DiffServ (0–5), ECN (6–7)
Total Length	Entire datagram size (header + data) in bytes
Identification	Unique ID for fragmentation/reassembly
Flags & Fragment Offset	For splitting and reassembling datagrams
TTL (Time To Live)	Max hops a packet can traverse, decremented at each router
Protocol	Indicates upper-layer protocol (TCP = 6, UDP = 17)
Header Checksum	Error detection for the header
Source IP	32-bit sender address
Destination IP	32-bit receiver address
Options	Optional features (timestamp, record route, etc.)

Important Notes

- **Overhead:** IP adds ~20 bytes; TCP adds another 20 bytes → total overhead 40 bytes.
- **Max Datagram Size:** 65,535 bytes (64K), but most networks use ≤1500 bytes for efficiency.

- **TTL:** prevents packets from circulating forever.

💡 **Analogy:**
Think of an IP datagram as a **postal package**:

- **Source & Destination IP:** sender & receiver addresses.
- **TTL:** expiration date to prevent lost packages lingering.
- **TOS:** priority label ("rush" vs "standard").
- **Fragmentation info:** if the package is too big for the road, split it and number the pieces.

📌 IP Fragmentation & Reassembly

1 Why Fragmentation Happens

- Every **network link** has a **Maximum Transmission Unit (MTU)** → the largest frame size it can carry.
- **Different links** have different MTUs.
- If an IP datagram is **larger than the MTU** of the next link, it must be **split** into smaller datagrams (fragments).

---

2 How Fragmentation Works

- **Original datagram:** 1 large IP packet.
- **Fragmentation process:** breaks it into **smaller datagrams**, each small enough to fit the link's MTU.
- **Each fragment has:**
  - Same **Identification** field → links fragments to the original datagram.
  - **Fragment Offset** → indicates the position of this fragment in the original datagram.
  - **More Fragments (MF) flag** → set to 1 if more fragments follow, 0 for the last fragment.

---

3 Reassembly at Destination

- **Fragments travel independently** → may take different paths.
- **Destination host** collects fragments using **Identification + Fragment Offset**.
- Once all fragments are received → **original datagram is reconstructed**.
- **Routers** do NOT reassemble fragments → only **destination host** does.

---

4 Example

- **Original datagram:** 4000 bytes
- **Link MTU:** 1500 bytes
- **Fragmentation result:** 3 smaller datagrams:
    0. Fragment 1 → 1500 bytes
    1. Fragment 2 → 1500 bytes
    2. Fragment 3 → 1000 bytes
- Each fragment carries **ID + offset + MF flag**.
- Destination reassembles → original 4000-byte datagram.

💡 **Analogy:**

- Imagine sending a large book through a postal service that only allows small packages.
- You **split the book into smaller packages**, label each with a sequence number, and send them.
- The recipient **reassembles the book** in the correct order.

📌 IP Address: Introduction

What is an IP Address?

- A **32-bit identifier** assigned to each **host or router interface**.
- **Interface:** connection between a host/router and a physical link.
- **Routers** usually have **multiple interfaces**.
- **Hosts** typically have **1–2 interfaces** (e.g., Ethernet, Wi-Fi).
- IP addresses are **associated with each interface**, not the device as a whole.

Format

- **Written in dotted-decimal notation**: 4 numbers (0–255) separated by dots.
- Example: `223.1.1.1` → binary: `11011111 00000001 00000001 00000001`

Examples of IP addresses

- `223.1.1.1`
- `223.1.1.2`
- `223.1.1.3`
- `223.1.1.4`
- `223.1.2.1`
- `223.1.2.2`

- `223.1.2.9`
- `223.1.3.1`
- `223.1.3.2`
- `223.1.3.27`

💡 **Analogy:** Think of an IP address like a **home address**. Each interface (door) gets its own address so mail (packets) can be delivered correctly.

📌 IP Addressing & Interfaces

Example IPs

- `223.1.1.1`, `223.1.1.2`, `223.1.1.3`, `223.1.1.4`
- `223.1.2.1`, `223.1.2.2`, `223.1.2.9`
- `223.1.3.1`, `223.1.3.2`, `223.1.3.27`

Q: How are these interfaces physically connected?

- **Wired:** Ethernet interfaces → connected via **Ethernet switches**
- **Wireless:** WiFi interfaces → connected via **WiFi base stations**

💡 For now: You **don't need to worry** about the exact physical connections between interfaces that are directly connected (without a router in between).

📌 IP Addressing

An **IP address** is a unique **32-bit identifier** for each host or router interface.

1️⃣ Binary Notation

- Each of the 4 octets (8 bits) is written in **binary**.
- Example:
- `223.1.1.1 → 11011111.00000001.00000001.00000001`

2️⃣ Hexadecimal Notation

- Each octet is converted to **hexadecimal**.
- Example:
- `223.1.1.1 → DF.01.01.01`

  (D=13, F=15 in decimal → 223 in decimal)

☑ **Tip for Students:**

- **Binary** is useful for subnetting and masks.
- **Hexadecimal** is shorter and sometimes used in networking tools or debugging.

IP addressing

- Notation
  - Binary Notation:
    - IP address is displayed as 32 bits.
    - 01110101 10010101 00011101 11101010
  - Dotted-Decimal Notation
    - IP addresses are written in decimal form with a decimal point (dot) separating the bytes.
    - 128.11.3.31
  - Hexadecimal Notation



Binary Notation

0111 0101 1001 0101 0001 1101 1110 1010

75 95 1D EA

0x75951DEA

Hexadecimal Notation

IP addressing

Find the error, if any, in the following IPv4 addresses.

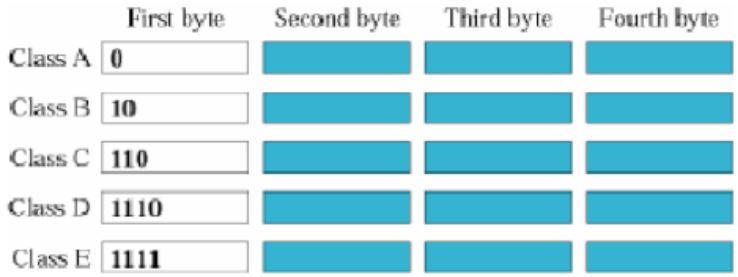
a)111.56.045.78
b)221.34.7.8.20
c)75.45.301.14
d)11100010.23.14.67

Solution

a)There must be no leading zero (045).
b)There can be no more than four numbers.
c)Each number needs to be less than or equal to 255.
d) A mixture of binary notation and dotted-decimal notation is not allowed.

IP addressing (Classful ~~Addressing~~)

- IP addresses were divided into 5 classes: A,B,C,D and E
  - This is the original scheme known as classful addressing
  - From mid-90's, classless addressing is introduced
  - However, classful addressing is still used



IP addressing (Classful ~~Addressing~~)

Exercise:

1. Find the class of these IP addresses:
   a) 11000001 10000011 00011011 11111111
   b) 10000001 10000011 00011011 11111111
2. How many class B addresses are there altogether?
3. What is the range of class B addresses? Answer this by giving the first and last class B addresses in dotted decimal notation.

IP addressing (Classful Addressing)

Solution:

1. a) First 3 bits are 110 -> Class C.
   b) First 2 bits are 10 -> Class B.
2. Class B addresses: the first two bits are 10 then followed by 30 bits of 1/0
   -> 2^30 addresses
3. The first and last class B addresses in binary are:

10000000 00000000 00000000 00000000

10111111 11111111 11111111 11111111

In dotted decimal notation, they are:

128.0.0.0 and 191.255.255.255

Network Layer 4-35

IP addressing (Classful Addressing)

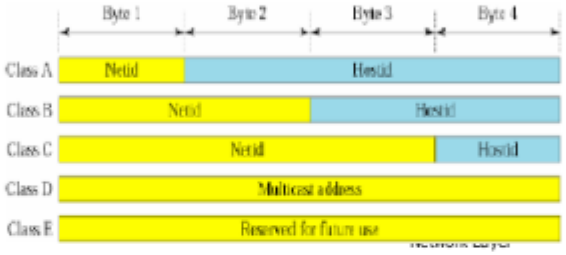
Netid and Hostid

- IP addresses in classes A,B and C are divided into netid and hostid
  - Netid: Identifying the network
  - Hostid: Identifying a host within the network
- Hosts within a network
  - Have the same netid
  - But different hostid

IP addressing (Classful Addressing)

Finding the class in decimal notation

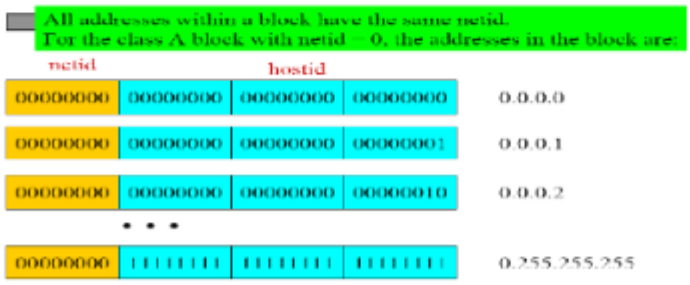




4-37

IP addressing (Classful Addressing)

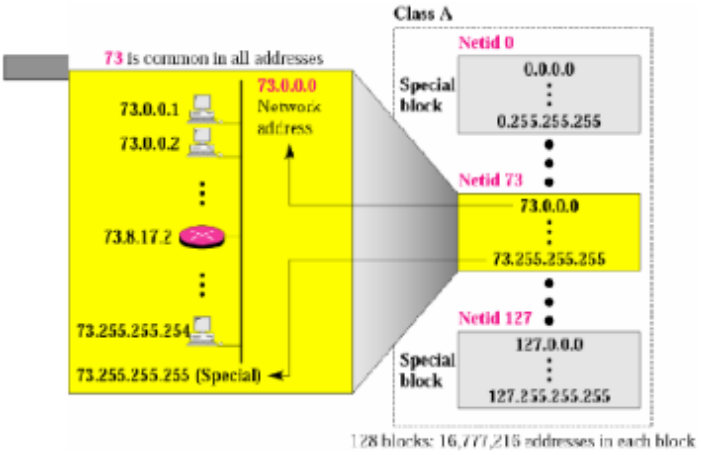
Classes and Blocks



- *Class A* is divided into 128 blocks
  - Each block has a different netid
  - 1st block: 0.0.0.0 to 0.255.255.255 (netid = 0)
  - 2nd block: 1.0.0.0 to 1.255.255.255 (netid = 1)
  - Last block: 127.0.0.0 to 127.255.255.255 (netid = 127)
- *Network address*: the first address of the block

IP addressing (Classful ~~Addressing~~)

Blocks in Class A

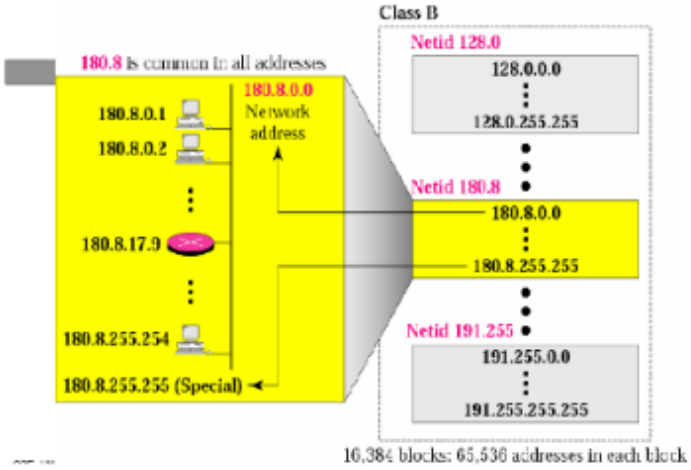


128 blocks: 16,777,215 addresses in each block

Network Layer 4-39

IP addressing (Classful ~~Addressing~~)

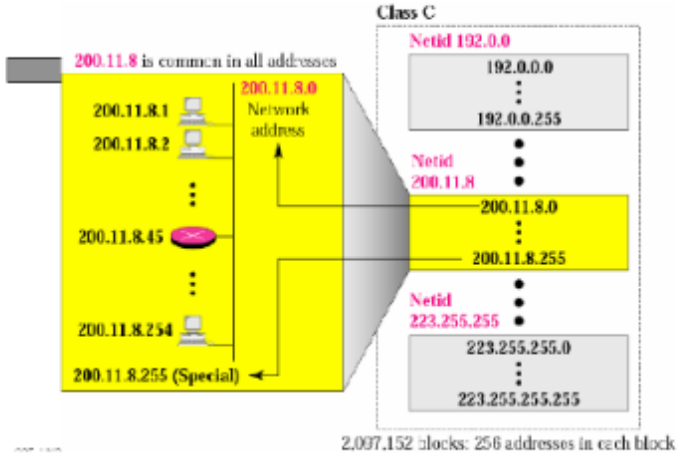
Blocks in Class B



16,384 blocks: 65,536 addresses in each block

IP addressing (Classful Addressing)

Blocks in Class C



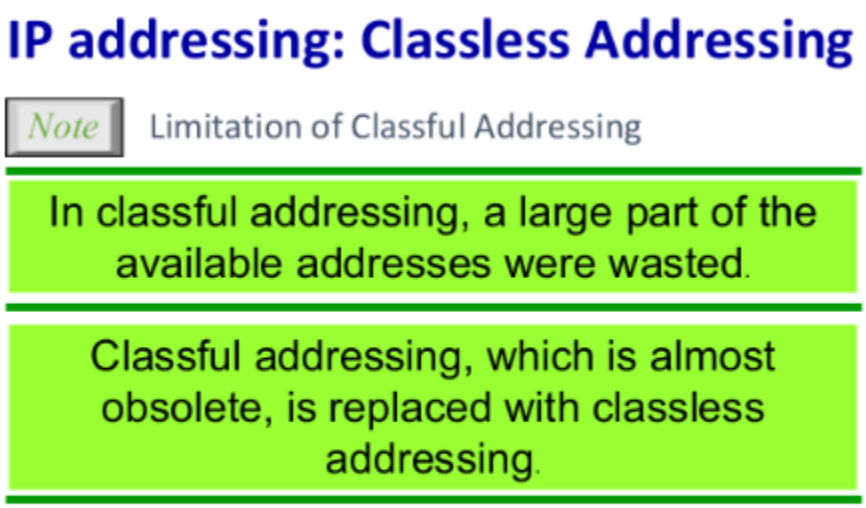
IP addressing (Classful Addressing)

Uses of Addresses

- Classes A, B and C addresses can be assigned to hosts, router ports etc
  - They are also known as unicast addresses
- Class D addresses are for multicast
  - Multicast: One sender, multiple recipients
- Class E addresses are reserved for special purposes

Network Addresses

- The network address is the first address in the block
- The network address defines the network to the rest of the Internet
  - Routers route packets based on network address
- Given the network address, we can find the class of the address and the range of the address in the block



📌 CIDR (Classless Inter-Domain Routing)

- Before CIDR, IP addresses were divided into **fixed classes (A, B, C)**. This wasted address space.
- **CIDR removes fixed classes** → instead, it allows **flexible subnet masks**.

🔹 CIDR Notation

- Format: **a.b.c.d/x**
- `/x` = number of **subnet bits** (network portion).
- Remaining bits = **host portion**.

Example:

```
200.23.16.0/23
```

- `/23` → first **23 bits** are the subnet (network) portion.
- Remaining **9 bits** are for hosts.
- Total usable hosts = 2^9 – 2 = 510 hosts.

📌 Subnets

- A **subnet** = a group of devices that can communicate **directly**, without passing through a router.
- All devices in a subnet **share the same subnet prefix** (network portion).
- The **host portion** is unique for each device.

Example with 3 subnets:

- Subnet 1: 223.1.1.0/24
- Subnet 2: 223.1.2.0/24
- Subnet 3: 223.1.3.0/24

☑ **Quick Way to Remember:**

- `/x` tells you how many bits are "locked" for the network.
- Fewer host bits = fewer hosts, more subnets.
- More host bits = more hosts, fewer subnets.

📘 Subnets (with a /24 mask)

🔧 Recipe for Defining Subnets

1. **Detach each interface** from its host/router → you get separate "islands."
2. **Each island = one subnet.**
3. **Choose a subnet mask.** Here it's **/24** → the **first 24 bits (first 3 octets)** are the subnet (network) part; the **last 8 bits** are the host part.

- With **/24**, any addresses that share the same first three octets (e.g., `223.1.3.*`) are in the **same subnet**.
- For a /24:
  - **Network address:** `a.b.c.0`
  - **Broadcast address:** `a.b.c.255`
  - **Usable hosts:** `a.b.c.1` through `a.b.c.254`

🧿 "Where are the subnets?" (for /24)

Group the given IPs by their first three octets:

🟩 Subnet 223.1.1.0/24

Members shown:

- 223.1.1.1, 223.1.1.2, 223.1.1.3, 223.1.1.4
  (Network ID would be **223.1.1.0**, broadcast **223.1.1.255**)

🟦 Subnet 223.1.2.0/24

Members shown:

- 223.1.2.1, 223.1.2.2, 223.1.2.6
  (Network ID **223.1.2.0**, broadcast **223.1.2.255**)

🟧 Subnet 223.1.3.0/24

Members shown:

- 223.1.3.1, 223.1.3.2, 223.1.3.27
  (Network ID **223.1.3.0**, broadcast **223.1.3.255**)

🟪 Subnet 223.1.7.0/24

Members shown:

- **223.1.7.0** (this is the **network address**, not a host),
- 223.1.7.1 (a usable host)
  (Broadcast would be **223.1.7.255**)

🟥 Subnet 223.1.8.0/24

Members shown:

- **223.1.8.0** (network address),
- 223.1.8.1 (usable host)
  (Broadcast **223.1.8.255**)

⬛ Subnet 223.1.9.0/24

Members shown:

- 223.1.9.1, 223.1.9.2
  (Network **223.1.9.0**, broadcast **223.1.9.255**)

🧠 Key Things to Notice (for /24)

- **Same first 3 octets → same subnet.**
- Addresses ending in **.0** are typically the **network ID** (not assignable to hosts).
- Addresses ending in **.255** are typically the **broadcast** (not assignable to hosts).
- Usable hosts are **.1–.254** within each /24.

☑ Quick Check

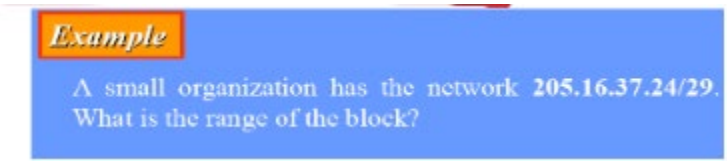
- Is **223.1.7.0** a host? → **No**, it's the network address of **223.1.7.0/24**.
- Are **223.1.1.3** and **223.1.1.4** in the same subnet (with /24)? → **Yes** (both in **223.1.1.0/24**).
- Do **223.1.2.6** and **223.1.3.2** belong to the same subnet (/24)? → **No** (they're in **223.1.2.0/24** and **223.1.3.0/24** respectively).

🔹 CIDR & Subnet Mask

- **CIDR (Classless InterDomain Routing)**: Written as `IP / prefix-length`.
  - Example: `172.16.4.0/24` → `/24` means **first 24 bits = network portion**, remaining **8 bits = host portion**.
- **Subnet Mask**:
  - Same thing, but written in **binary/decimal**.
  - `/24` = `11111111.11111111.11111111.00000000` → `255.255.255.0`.
  - `/29` = `11111111.11111111.11111111.11111000` → `255.255.255.248`.

👉 **Rule:** Prefix length = number of 1's in the subnet mask.

🔹 Example Walkthrough (Your case: `/29`)



Given address:

```
11001111 00010000 00100101 00011000
= 207.16.37.24  (in decimal)
```

1. **Prefix length:** `/29` → means **first 29 bits = network part**, remaining **3 bits = host part**.
2. **Block size:** `2^(32-29) = 2^3 = 8`.
   - So, each subnet has **8 addresses**.
3. **First address (Network ID):**
   - Set last 3 bits to `000`:
   - `11001111 00010000 00100101 00011000` = `207.16.37.24`.
4. **Last address (Broadcast):**
   - Set last 3 bits to `111`:
   - `11001111 00010000 00100101 00011111` = `207.16.37.31`.
5. **Range of valid host addresses:**
   - Between first+1 and last−1 → `207.16.37.25` to `207.16.37.30`.

☑ **Final Answer (CIDR Block 207.16.37.24/29):**

- **Subnet Mask:** `255.255.255.248`
- **Block size:** 8 addresses
- **Network ID:** `207.16.37.24`
- **Broadcast Address:** `207.16.37.31`
- **Usable Hosts:** `207.16.37.25 - 207.16.37.30` (6 usable)

What is the network address if one of the addresses is 167.199.170.82/27?

We are given: **167.299.170.82/27**

Step 1: Understand `/27`

- `/27` means **first 27 bits = network portion**, last `32-27 = 5 bits = host portion`.
- Block size = `2^5 = 32 addresses` per subnet.

Step 2: Find the block range

- Subnets increase in **steps of 32** in the last octet (because host portion = 5 bits).
- Last octet of given IP = `82`.

Now check multiples of 32 around 82:

- `32, 64, 96, 128…`

`82` lies between **64 and 96**.

Step 3: Network Address

- Always the **lowest address in the block**.
- So, **167.299.170.64**

Step 4: Broadcast Address

- Always the **highest address in the block**.

- So, **167.299.170.95**

Step 5: Usable Hosts

- From `167.299.170.65` → `167.299.170.94` (30 usable addresses).

☑ **Final Answer:**

- **Network Address:** `167.299.170.64`
- **Broadcast Address:** `167.299.170.95`
- **Usable Range:** `167.299.170.65 – 167.299.170.94`

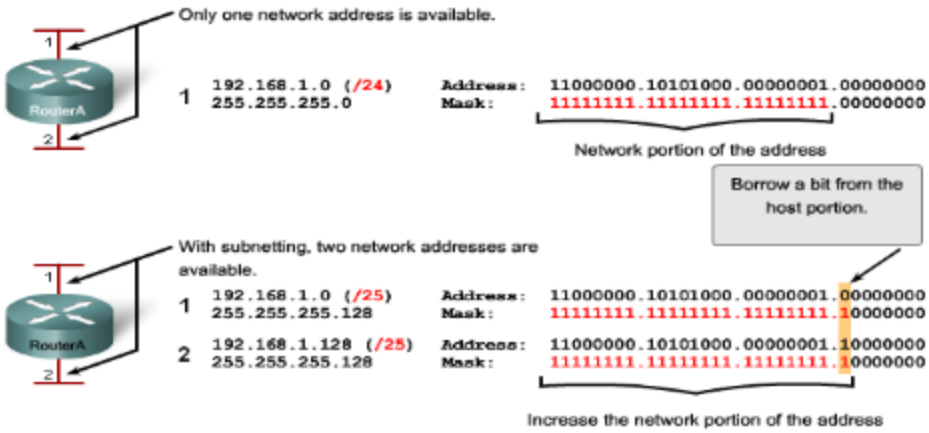
IP Addressing (CIDR): Subnetting

- **Why subnet?**
  - To split one large network into **smaller logical networks** (subnets).
  - Helps in better **management, security, and efficient IP usage**.

- **How is it done?**
  0. Start with an **IP block** (e.g., `192.168.1.0/24`).
  1. Use some of the **host bits** and turn them into **network bits**.
  2. This is called **borrowing bits**.
  3. The subnet mask is extended to reflect the new division.

- **Effect of subnetting**
  - **More subnets** are created.
  - **Fewer hosts per subnet** (since host bits are reduced).
  - Example:
    - `/24` → 256 total addresses (1 subnet, 254 usable hosts).
    - Borrow 2 bits → `/26` → 4 subnets, each with 64 addresses (62 usable hosts).

☑ **In short:**
Subnetting = **borrowing host bits → creating more subnets but fewer hosts per subnet**.



📌 CIDR: Calculating Subnets and Hosts

1. Number of Subnets

- Formula: $\mathbf{2^n}$
- Where **n = number of bits borrowed** (from the host portion).
- Example: borrow **1 bit** → 2^1 = **2 subnets**
  borrow **3 bits** → 2^3 = **8 subnets**

👉 The more bits you borrow, the **more subnets** you create.

---

2. Number of Usable Hosts per Subnet

- Formula: $\mathbf{2^h - 2}$
- Where **h = number of host bits left**.
- Why minus 2?
  - One address = **network address**
  - One address = **broadcast address**
- Example:
  - If **7 host bits left** → $\mathbf{2^7 - 2 = 128 - 2 = 126}$ **usable hosts**.
  - If **5 host bits left** → $\mathbf{2^5 - 2 = 32 - 2 = 30}$ **usable hosts**.

👉 The more bits you borrow, the **fewer hosts** you can have per subnet.

🔑 Key Trade-off:

- **Borrow more bits → more subnets, fewer hosts per subnet.**
- **Borrow fewer bits → fewer subnets, more hosts per subnet.**

🔹 Case: n = 0

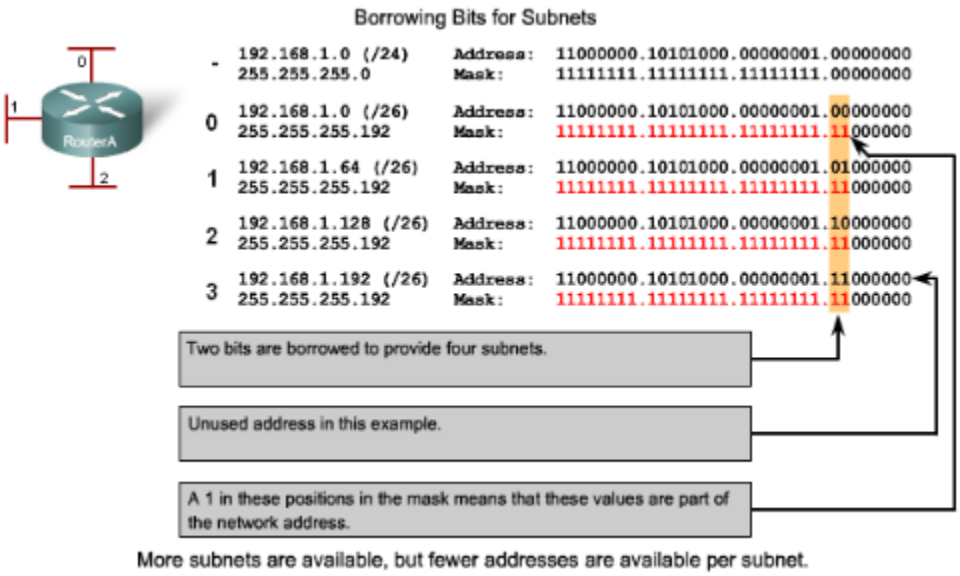
- **Number of subnets = $2^0 = 1$**
  👉 Means we have **only 1 network** (the original network).
- **Host bits (h) = whatever was originally assigned.**
  For example, in a Class C block (`/24`), there are **8 host bits**.
  - Usable hosts = $2^8 - 2$ = **254 hosts**

☑ So if n = 0:

- You don't split into smaller subnets.
- You just have the original big network with all its hosts.

◇ Example 1: Borrow 2 host bits

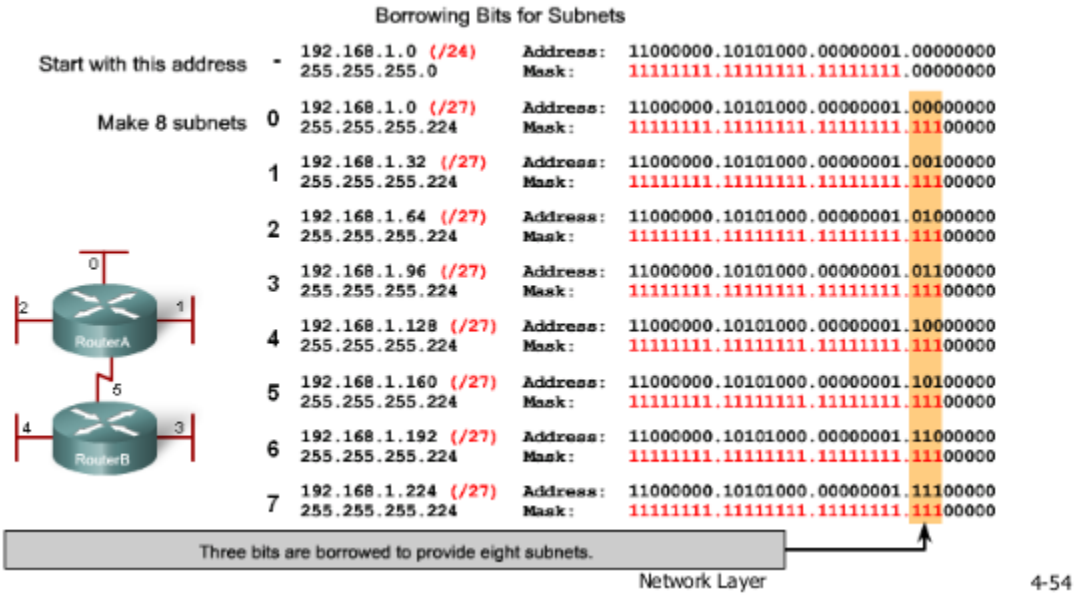
- Suppose we start with: **192.168.1.0/24**
  (Here `/24` means: 24 network bits, 8 host bits left).
- If we **borrow 2 host bits** → `/26` (24 + 2 = 26).
- Formula:
  - **Number of subnets = $2^n = 2^2 = 4$**
  - **Number of hosts per subnet = $2^h - 2 = 2^6 - 2 = 62$ usable hosts**
- Subnets created:
  - 192.168.1.0/26 → hosts: 1–62, broadcast: .63
  - 192.168.1.64/26 → hosts: 65–126, broadcast: .127
  - 192.168.1.128/26 → hosts: 129–190, broadcast: .191
  - 192.168.1.192/26 → hosts: 193–254, broadcast: .255



☑ **Result:** 4 subnets, each with 62 usable hosts.

◇ Example 2: Borrow 3 host bits

- Start again with: **192.168.1.0/24**
- If we **borrow 3 host bits** → `/27` (24 + 3 = 27).
- Formula:
  - **Number of subnets = $2^3 = 8$**
  - **Number of hosts per subnet = $2^5 - 2 = 30$ usable hosts**
- Subnets created:
  - 192.168.1.0/27 → hosts: 1–30, broadcast: .31
  - 192.168.1.32/27 → hosts: 33–62, broadcast: .63
  - 192.168.1.64/27 → hosts: 65–94, broadcast: .95
  - 192.168.1.96/27 → hosts: 97–126, broadcast: .127
  - 192.168.1.128/27 → hosts: 129–158, broadcast: .159
  - 192.168.1.160/27 → hosts: 161–190, broadcast: .191
  - 192.168.1.192/27 → hosts: 193–222, broadcast: .223
  - 192.168.1.224/27 → hosts: 225–254, broadcast: .255



☑ **Result:** 8 subnets, each with 30 usable hosts.

💡 **Key takeaway:**

- Borrowing more bits → **more subnets** ☑
- But fewer host addresses per subnet ❌

Subnetting Classless Network

- A classless network also be subnetted
- Example:

  An organization is granted the block 130.34.12.64/26. The organization needs to have four subnets. What are the subnet addresses and the range of addresses for each subnet?

Text book

IP addressing (CIDR)

Solution

Since the prefix length is 26. This means the last 6 bits are available as hostid, the total number of addresses in the block is 64 (2^6). If we create four subnets, each subnet will have 16 addresses.

Solution (Continued)

Let us first find the subnet prefix (subnet mask). We need four subnets, which means we need to add two more 1s to the site prefix. The subnet prefix is then /28.

Subnet 1: 130.34.12.64/28 to 130.34.12.79/28.

Subnet 2 : 130.34.12.80/28 to 130.34.12.95/28.

Subnet 3: 130.34.12.96/28 to 130.34.12.111/28.

Subnet 4: 130.34.12.112/28 to 130.34.12.127/28.

Solution:

The binary representation of the given address 130.34.12.64/26 is

10001000 00100010 00001100 01000000

The organization need to have 4 subnets so, need to borrow a minimum of 2 host bits.

2^2= 4 subnets

Subnet 1:

First address: 10001000 00100010 00001100 01000000 or 130.34.12.64/28

Last address: 10001000 00100010 00001100 01001111 or 130.34.12.79/28

Range: 130.34.12.64/28 - 130.34.12.79/28

Subnet 2:

First address: 10001000 00100010 00001100 01010000 or 130.34.12.80/28

Last address: 10001000 00100010 00001100 01011111 or 130.34.12.95/28

Range: 130.34.12.80/28 - 130.34.12.95/28

Subnet 3:

First address: 10001000 00100010 00001100 01100000 or 130.34.12.96/28

Last address: 10001000 00100010 00001100 01101111 or 130.34.12.111/28

Range: 130.34.12.96/28 - 130.34.12.111/28

Subnet 4:

First address: 10001000 00100010 00001100 01110000 or 130.34.12.112/28

Last address: 10001000 00100010 00001100 01111111 or 130.34.12.127/28

Range: 130.34.12.112/28 - 130.34.12.127/28

So, the range of the addresses for each subnet is $2^{32-28} = 2^4 = 16$

Text book

IP addressing (CIDR)

An ISP is granted a block of addresses starting with 190.100.0.0/16 (65,536 addresses). The ISP needs to distribute these addresses to three groups of customers as follows:

a. The first group has 64 customers; each needs 256 addresses.

b. The second group has 128 customers; each needs 128 addresses.

c. The third group has 128 customers; each needs 64 addresses.

Design the subblocks and find out how many addresses are still available after these allocations.

Group 1

For this group, each customer needs 256 addresses. This means that 8 bits are needed to define each host. The prefix length is then 32 − 8 = 24. The addresses are

1st Customer:	*190.100.0.0/24*	*190.100.0.255/24*
2nd Customer:	*190.100.1.0/24*	*190.100.1.255/24*
. . .		
64th Customer:	*190.100.63.0/24*	*190.100.63.255/24*

Total = 64 × 256 = 16,384

Group 2
For this group, each customer needs 128 addresses. This means that 7 bits are needed to define each host. The prefix length is ther 32 − 7 = 25. The addresses are

1st Customer:	*190.100.64.0/25*	*190.100.64.127/25*
2nd Customer:	*190.100.64.128/25*	*190.100.64.255/25*
. . .		
128th Customer:	*190.100.127.128/25*	*190.100.127.255/25*

Total = 128 × 128 = 16,384

Group 3
For this group, each customer needs 64 addresses. This means that 6 bits are needed to each host. The prefix length is then 32 − 6 = 26. The addresses are

1st Customer:	*190.100.128.0/26*	*190.100.128.63/26*
2nd Customer:	*190.100.128.64/26*	*190.100.128.127/26*
. . .		
128th Customer:	*190.100.159.192/26*	*190.100.159.255/26*

Total = 128 × 64 = 8192

Number of granted addresses to the ISP: 65,536
Number of allocated addresses by the ISP: 40,960
Number of available addresses: 24,576

◇ Two Ways a Host Gets an IP Address

1. **Manual (Static configuration)**
   - System admin types in the IP address by hand.
   - Example:
     - Windows: *Control Panel → Network → TCP/IP Properties*
     - UNIX/Linux: */etc/rc.config*
   - Good for servers (they need a fixed, permanent IP).
   - ❌ Problem: Not flexible. If you move the device or network changes, you may need to reconfigure.
2. **Automatic (Dynamic configuration, DHCP)**
   - **DHCP (Dynamic Host Configuration Protocol)** gives IP addresses automatically.
   - Known as **"plug-and-play"**.
   - Good for laptops, phones, visitors — devices that join and leave networks.
   - ☑ Address leases (temporary use) → IPs can be reused.

◇ DHCP Working Process (Step by Step)

When a host connects to the network:

1. **DHCP Discover** → Host broadcasts: "Is there a DHCP server?"
2. **DHCP Offer** → Server replies: "I can give you this IP."
3. **DHCP Request** → Host says: "I'd like to use that IP."
4. **DHCP ACK** → Server confirms: "Okay, that IP is yours (for a lease time)."

This whole exchange is sometimes remembered as **DORA**:

- **D**iscover
- **O**ffer
- **R**equest
- **A**ck

☑ **In summary:**

- Static IP = fixed, set manually (e.g., servers).
- DHCP = automatic, dynamic, temporary (e.g., laptops, phones).

◇ DHCP Client–Server Example

- **DHCP Server Address:** `223.1.2.5` (inside the router, serving subnets)
- **New Client:** just arrived, needs an IP in `223.1.2.x` network

---

1. DHCP Discover (Client → Broadcast)

- **Source IP:** `0.0.0.0` (client has no IP yet)
- **Dest IP:** `255.255.255.255` (broadcast → all devices)
- **Port:** client uses `68`, server uses `67`
- **Transaction ID:** 654
- **yiaddr (Your IP Address):** `0.0.0.0` (empty for now)

👉 Client is basically shouting:
"Hey, is there any DHCP server? I need an IP!"

---

2. DHCP Offer (Server → Broadcast)

- **Source IP:** `223.1.2.5` (DHCP server)
- **Dest IP:** `255.255.255.255` (broadcast)
- **yiaddr:** `223.1.2.4` (server suggests this IP)
- **Transaction ID:** 654 (matches client's discover)

- **Lease Time:** 3600 sec (1 hour)

👉 Server replies:
"Yes! I can give you 223.1.2.4 for 1 hour."

---

3. DHCP Request (Client → Broadcast)

- **Source IP:** `0.0.0.0` (still no confirmed IP)
- **Dest IP:** `255.255.255.255`
- **yiaddr:** `223.1.2.4` (the offered IP)
- **Transaction ID:** 655 (new one)
- **Lease Time:** 3600 sec

👉 Client asks:
"I'd like to use the IP 223.1.2.4, please confirm."

---

4. DHCP ACK (Server → Broadcast)

- **Source IP:** `223.1.2.5` (DHCP server)
- **Dest IP:** `255.255.255.255`
- **yiaddr:** `223.1.2.4`
- **Transaction ID:** 655
- **Lease Time:** 3600 sec

👉 Server confirms:
"Okay! 223.1.2.4 is now officially yours."

☑ **Summary (DORA process):**

- Discover → Offer → Request → ACK
- Client successfully gets IP: `223.1.2.4`

🔹 DHCP Client–Server Example (DORA Process)

DHCP Server: 223.1.2.5 (inside router, serving all subnets)
Client: newly arrived device, needs an IP in `223.1.2.x`

1️⃣ DHCP Discover (Client → Broadcast)

- **Src IP:** `0.0.0.0` (no IP yet)
- **Dest IP:** `255.255.255.255` (broadcast to all)
- **Ports:** client uses 68, server listens on 67
- **yiaddr:** `0.0.0.0`

👉 Client shouts:
"Is there a DHCP server? I need an IP!"

2️⃣ DHCP Offer (Server → Broadcast)

- **Src IP:** `223.1.2.5`
- **Dest IP:** `255.255.255.255`
- **yiaddr:** `223.1.2.4` (offered IP)
- **Lease:** 3600 sec

👉 Server replies:
"Yes, you can use `223.1.2.4` for 1 hour."

3️⃣ DHCP Request (Client → Broadcast)

- **Src IP:** `0.0.0.0`
- **Dest IP:** `255.255.255.255`
- **yiaddr:** `223.1.2.4` (asking for the offered IP)

👉 Client says:
"I want to use `223.1.2.4`, please confirm!"

4️⃣ DHCP ACK (Server → Broadcast)

- **Src IP:** `223.1.2.5`
- **Dest IP:** `255.255.255.255`
- **yiaddr:** `223.1.2.4`
- **Lease:** 3600 sec

👉 Server confirms:
"OK! `223.1.2.4` is officially yours."

☑ **Summary (DORA):**

- **D**iscover → **O**ffer → **R**equest → **A**CK
- Final IP: **223.1.2.4**

✨ To remember: DHCP is **plug-and-play** → you just join the network, and this 4-step handshake gives you an IP without manual setup.

🔹 What is NAT?

- NAT is like a **middleman** between your **private network** (home/office LAN) and the **public Internet**.
- Inside your home, devices (PC, phone, printer) use **private IPs** like:
  - `10.x.x.x`
  - `172.16.x.x – 172.31.x.x`
  - `192.168.x.x`
- But the **Internet doesn't allow private IPs**.
  So NAT makes them all look like **one public IP** when they go online.

🔹 Example Scenario

- Local network: `10.0.0.0/24`
- Public NAT IP (from ISP): `138.76.29.7`

👉 Suppose three devices inside LAN send packets:

- Laptop (10.0.0.2 → Google.com)
- Phone (10.0.0.3 → YouTube.com)
- Printer (10.0.0.4 → Cloud service)

When these packets leave:

- All appear to come from **138.76.29.7** (same public IP).
- NAT router assigns a **different port number** to each connection to keep track.

🔹 Key Idea: Port Mapping

- NAT keeps a **translation table**:
- `Inside (private)  Outside (public)`

- `10.0.0.2:3001` ↔ `138.76.29.7:4001`
- `10.0.0.3:3002` ↔ `138.76.29.7:4002`
- `10.0.0.4:3003` ↔ `138.76.29.7:4003`
- This way, when a reply comes back to `138.76.29.7:4002`, NAT knows it belongs to **10.0.0.3:3002** (the phone).

🔹 Advantages of NAT

1. **Saves IPv4 addresses** – only one public IP needed for the whole LAN.
2. **Flexibility** – you can change local IPs without telling the ISP.
3. **Easier ISP change** – just update the NAT IP, not all devices.
4. **Security** – outside hosts cannot directly reach your private devices.

☑ **In short:**
NAT lets many private devices share **one public IP** using port mapping, solving IPv4 address shortage and adding a security layer.

🔹 How NAT Router Works (Step by Step)

1. **Outgoing Datagrams (LAN → Internet)**
   - Suppose your laptop (10.0.0.2:3001) sends a packet to Google.
   - NAT router **modifies** the packet header:
     - **Source IP**: changes from `10.0.0.2` → `138.76.29.7` (public IP).
     - **Source Port**: changes from `3001` → `4001` (new unique port).
   - NAT records this in its **translation table**:
   - `10.0.0.2:3001 → 138.76.29.7:4001`
2. **Remote Server Response (Internet → LAN)**
   - Google replies to `138.76.29.7:4001`.
   - NAT router checks its translation table.
   - It sees that `4001` is mapped to `10.0.0.2:3001`.
   - NAT **replaces destination info**:
     - **Dest IP**: `138.76.29.7` → `10.0.0.2`.
     - **Dest Port**: `4001` → `3001`.
   - Packet gets forwarded to your laptop. ☑

🔹 NAT Translation Table Example

Private (LAN) Public (Internet)

10.0.0.2:3001 138.76.29.7:4001

10.0.0.3:3002 138.76.29.7:4002

10.0.0.4:3003 138.76.29.7:4003

🔹 Key Points

- NAT must **transparently rewrite headers** (IP + Port).
- Every outgoing connection gets a **unique public port**.
- Every incoming packet is checked against the **translation table**.
- If no entry exists → NAT **drops the packet** (protecting LAN).

👉 Think of NAT like a **receptionist** at an office:

- When employees (private IPs) make phone calls, the receptionist shows the office's **main number (public IP)** but keeps a record of **which extension (port)** belongs to which employee.
- When calls come back, the receptionist routes them to the correct extension.

🔹 Scenario

- **Private LAN IPs**: 10.0.0.1, 10.0.0.2, 10.0.0.3, 10.0.0.4
- **NAT router's public IP**: 138.76.29.7
- **Remote server**: 128.119.40.186 (running web service on port 80)

🔹 Step-by-Step Process

Step 1 – Outgoing Packet

- Host `10.0.0.1` sends a packet to server:
- `Source: 10.0.0.1, 3345`
- `Destination: 128.119.40.186, 80`
- Port **3345** is the host's random **ephemeral port**.

👉 Problem: `10.0.0.1` is a **private IP**, not routable on the Internet.

Step 2 – NAT Translation

- NAT router rewrites the packet header:
- `Source: 138.76.29.7, 5001`
- `Destination: 128.119.40.186, 80`
- At the same time, it creates a **mapping in the NAT table**:
- `138.76.29.7:5001 <–> 10.0.0.1:3345`

👉 Now the packet looks valid to the outside world (uses NAT's public IP).

Step 3 – Incoming Reply

- Server replies to NAT's public address:
- `Source: 128.119.40.186, 80`
- `Destination: 138.76.29.7, 5001`
- From the server's perspective, it is talking to `138.76.29.7`.

Step 4 – NAT Reverse Translation

- NAT checks its table, sees `5001` maps to `10.0.0.1:3345`.
- It rewrites the destination back:
- `Source: 128.119.40.186, 80`
- `Destination: 10.0.0.1, 3345`
- Packet is forwarded inside LAN. ☑

🔹 NAT Translation Table Snapshot

Public (WAN) Side	Private (LAN) Side
138.76.29.7:5001	10.0.0.1:3345
138.76.29.7:5002	10.0.0.2:3370
138.76.29.7:5003	10.0.0.3:3441

Each host gets a unique mapping with **public port numbers**.

👉 So the **outside world only ever sees NAT's public IP (138.76.29.7)**.
👉 Inside, NAT secretly keeps track of which local host is using which connection.

🔹 ICMP (Internet Control Message Protocol)

- **Purpose**: IP itself doesn't provide error reporting or diagnostics, so ICMP is used to carry **control messages** about problems or status in the network.
- **Encapsulation**: ICMP messages are carried inside IP datagrams (so it sits just "above" IP).
- **Structure**:
  - **Type** (what kind of message)
  - **Code** (finer detail of the type)
  - **Data** (often includes the first 8 bytes of the IP packet that caused the error — to identify the problem packet)

🔹 Common ICMP Message Types

Type	Code	Meaning
0	0	Echo reply (used in **ping**)
3	0	Destination **network** unreachable
3	1	Destination **host** unreachable
3	2	Destination **protocol** unreachable
3	3	Destination **port** unreachable
3	6/7	Network/Host unknown
4	0	Source quench (old congestion control – **deprecated**)
8	0	Echo request (used in **ping**)
9	0	Route advertisement
10	0	Router discovery
11	0	**TTL expired** (important for traceroute)
12	0	Bad IP header

🔹 Tools that Use ICMP

☑ *Ping*

- Sends **ICMP Echo Request** (type 8).
- Destination replies with **ICMP Echo Reply** (type 0).
- Used to check **reachability** and measure **round-trip time (RTT)**.

☑ *Traceroute*

- Source sends a series of **UDP packets** with increasing TTL (Time-to-Live) values.
  - 1st probe → TTL = 1
  - 2nd probe → TTL = 2
  - …and so on.
- Each router where TTL expires discards the packet and sends back an **ICMP Time Exceeded (Type 11, Code 0)** message.
- Source records the **IP address and RTT** from each ICMP message → builds a hop-by-hop route map.
- Eventually, the packet reaches the destination host, which returns **ICMP Port Unreachable (Type 3, Code 3)** because the chosen port was invalid → traceroute stops.

👉 That's how traceroute reveals the **path packets take** across the Internet.

☑ Quick Check for You:
Why do you think traceroute uses **invalid UDP port numbers** at the end, instead of valid ones?

◇ IPv6 Datagram Format (simplified view)

Header fields (fixed 40 bytes):

- **Version (ver)** → always `6` for IPv6
- **Priority (pri)** → lets sources mark packets with different priorities (traffic class)
- **Flow Label** → marks packets as belonging to the same "flow" (like a video stream). (Still not well defined in practice.)
- **Payload Length** → length of the data (excluding header)
- **Next Header** → identifies upper-layer protocol (like TCP = 6, UDP = 17, ICMPv6, etc.)
- **Hop Limit** → replaces IPv4's TTL, decremented at each router
- **Source Address** → 128-bit IPv6 source
- **Destination Address** → 128-bit IPv6 destination
- **Payload (data)**

👉 Compared to IPv4, IPv6 has a **simpler, fixed-length header** → faster processing.

◇ What's Missing (vs IPv4)?

- **No header checksum** → reduces processing delay at routers.
- **No fragmentation/reassembly** at routers → only the **end hosts** can fragment.
- **No options field** → handled by *extension headers* (if needed).

◇ Transition from IPv4 → IPv6

Problem: Not all routers can switch to IPv6 at once → we need a way for IPv6 traffic to travel across IPv4-only parts of the network.

☑ Solution: Tunneling

- Idea: Wrap (encapsulate) an **IPv6 datagram inside an IPv4 datagram**.
- To the IPv4 routers, it just looks like a normal IPv4 packet.
- At the other end of the tunnel, the IPv6 datagram is extracted and forwarded.

🔹 Example of Tunneling (your diagram notes)

1. **A → B**: IPv6 packet created at source (A).
2. **B → C**: Between two IPv6 routers connected via an IPv4-only network.
    - IPv6 datagram is encapsulated inside an IPv4 datagram → "packet within a packet."
    - Source = B, Dest = E in IPv4 header.
3. **E → F**: IPv4 header removed, IPv6 datagram continues natively.
4. Final destination (F) sees the original IPv6 packet (src=A, dest=F).

👉 **Logical view:** IPv6 packet travels end-to-end.
👉 **Physical view:** Inside IPv4 tunnels wherever the path doesn't support IPv6.

☑ Quick check for you:
Why do you think IPv6 designers **removed fragmentation at routers** (only hosts can fragment now)?

◇ Why Transition Is Needed

- Not all routers on the Internet can be upgraded to IPv6 at the same time.

- So we need a way for IPv6 packets to travel across **IPv4-only parts of the network**.
- Solution: **Tunneling** (IPv6 packet inside IPv4 packet, like a box inside another box).

◇ Tunneling & Encapsulation

1. **Normal case (same tech all the way):**
   - Ethernet between IPv6 routers → IPv6 datagram just goes as payload in the link-layer frame.
   - Simple: Ethernet frame → IPv6 datagram → data.
2. **Mixed case (IPv6 routers separated by IPv4 network):**
   - Two IPv6 routers (B and E) are connected through an IPv4 network in the middle.
   - IPv6 router B takes the IPv6 datagram, **wraps it inside an IPv4 datagram**, and sends it across.
   - Router E removes the IPv4 "envelope" and continues forwarding the original IPv6 datagram.

👉 This is what we call **IPv4 tunnel**.

◇ Example Flow (your diagram with A → F)

- **A → B:** IPv6 packet created.
  - Flow X: Src=A, Dest=F, Data.
- **B → C:** IPv6 datagram **encapsulated inside IPv4 datagram**.
  - IPv4 header: src=B, dest=E (because B and E are IPv4 neighbors).
  - Inside payload: original IPv6 datagram (src=A, dest=F).
- **E → F:** At E, IPv4 header removed, pure IPv6 datagram continues to F.
  - Still Flow X: src=A, dest=F.

◇ Key Idea

- **Logical view:** From A to F, it looks like one continuous IPv6 path.
- **Physical view:** The packet sometimes has to "hide" inside IPv4 datagrams to get through IPv4-only regions.

💡 Think of it like mailing a **smaller box** inside a **bigger shipping box** if part of the transport system only accepts certain types of boxes.

☑ Quick check for you:
If an IPv6 datagram is tunneled inside IPv4, which IP addresses will the **IPv4 header** use? (the original A–F, or the addresses of the IPv4 tunnel endpoints like B and E?)